TestSplit Functional Specification

Authors: Samson Oloruntola & Marjia Siddik
Supervisor: Dr Paul Clarke
Module: CSC4098 - Final Year Project
Submission Date: 10/11/2025

Table of Contents

1. Introduction

1.1 Overview

TestSplit is an open-source tool that optimises Continuous Integration (CI) pipelines by analysing unit-test performance and generating optimised configuration files that distribute tests across parallel jobs. It integrates into existing development and CI environments with minimal setup and no infrastructure changes.

The tool uses the **Longest Processing Time (LPT)** scheduling algorithm, which aims to minimise total runtime by assigning longer tests first, allowing a more even workload across jobs. This process reduces waiting time without requiring changes to the tests themselves. Over time, TestSplit builds a history of test performance, helping teams detect slowdowns, identify regressions, and optimise their pipelines.

Key functions of the tool include:

- Parsing test results from JUnit XML file formats
- Profiling test execution times and storing results in a central database
- Generating optimised CI configuration files for GitHub Actions and GitLab CI
- Providing a command-line interface for profiling and configuration generation
- Displaying historical trends and performance metrics through a web dashboard

TestSplit integrates into CI workflows by generating configuration files that assign test groups to parallel jobs. It works with standard test output formats and requires no modifications to the test code or existing CI pipelines.

1.2 Business / Academic Context

This project is not sponsored by a specific business organisation, but it addresses a common and well-documented challenge in modern software development workflows: slow, inefficient Continuous Integration (CI) pipelines driven by large, unbalanced test suites. In many organisations, particularly those practising agile development or continuous delivery, long CI wait times can reduce developer productivity, slow feedback loops, and delay deployments.

TestSplit can be deployed to any software development team that relies on automated testing in CI pipelines. This includes startups, mid-size development teams, and large enterprises using CI/CD platforms such as GitHub Actions and GitLab CI, particularly those managing complex monorepos or legacy codebases where test suites have become performance bottlenecks.

By automatically profiling test performance and intelligently distributing tests across parallel jobs, TestSplit offers a practical solution to improve CI efficiency, reduce infrastructure costs, and support faster development cycles.

1.3 Glossary

Name	Description
LPT Algorithm	The Longest Processing Time scheduling algorithm is used to distribute tests across parallel jobs for optimal workload balance.
Baseline Run	The initial profiling execution that establishes reference timings for future comparisons and optimisations.
CI (Continuous Integration)	A software development practice where code changes are automatically built and tested to maintain integration stability.
CI Configuration	YAML-based configuration file that defines job matrices, parallel test runs, and build stages for CI platforms like GitHub Actions and GitLab CI.
CLI (Command-Line Interface)	The text-based interface through which users interact with TestSplit to profile tests and generate CI configurations.
Containerisation	The process of packaging software and dependencies into isolated environments (containers) for consistent execution.
Dashboard	The frontend web interface for visualising profiling results, job-distribution balance, and historical trends.
Dataset/Profile Data	Structured performance data (e.g., JSON) generated during profiling for reuse and trend analysis.
Docker	The container platform used by TestSplit to execute profiling tasks in consistent, reproducible environments.
Job/Job Group	A logical set of tests scheduled to run in parallel as part of a CI pipeline, generated by the LPT algorithm.
JUnit	A Java testing framework that produces XML-formatted results; these serve as input data for TestSplit's parser.
LPT Scheduler	Core module implementing the Longest Processing Time algorithm to determine balanced test groupings.
Parser Module	Component responsible for reading and interpreting JUnit XML files to extract test metadata and durations.
Profiling	The process of analysing test execution times to identify bottlenecks and produce data for optimisation.
Regression Detection	The identification of performance degradations or runtime imbalances across multiple profiling runs.
Speed-Up Factor	A calculated metric showing the improvement in CI runtime achieved through optimised test distribution.
Test Distribution	The allocation of test cases into balanced job groups to achieve minimal overall runtime.

TestSplit	The overall system that analyses test performance, balances workloads, and generates an optimised CI configuration
YAML File	Configuration file format used by CI systems to define workflows, runners, and parallel job setups.

2. General Description

2.1 Product Perspective

TestSplit is an open-source tool that optimises Continuous Integration (CI) pipelines by analysing unit-test performance and generating a pipeline configuration that distributes tests across parallel jobs. TestSplit integrates into existing development and CI environments, providing automation without requiring major infrastructure changes.

Local Development:

- Developers use the TestSplit CLI to analyse JUnit test results and create optimisation profiles.
- The CLI parses JUnit 4 and 5 XML output files to extract individual test durations and compute workload distributions using the Longest Processing Time (LPT) scheduling algorithm.
- Profiling runs are executed in a consistent, containerised environment (Docker) with fixed resource limits to ensure reproducible test timings and stable benchmarking.
- This approach allows developers to detect slow tests, measure performance improvements, and generate optimised CI configurations for use in their projects.

CI Pipelines:

- TestSplit integrates with platforms such as GitHub Actions and GitLab CI by generating ready-to-use YAML configuration files.
- These configurations define how tests are distributed across multiple parallel jobs based on measured execution times and the LPT-derived scheduling strategy.
- Once deployed, the CI platform executes tests in parallel according to the optimised configuration, reducing total pipeline runtime without modifying existing tests.

Dashboard:

- The TestSplit dashboard provides an interactive web interface for visualising profiling results, optimisation metrics, and the job distribution balance.
- It is implemented as a lightweight React application that loads data locally from CLI-generated JSON files, enabling developers to review performance trends and compare baseline versus optimised runs.
- The dashboard supports deployment to static hosting services such as Vercel, Netlify, or GitHub Pages.

2.2 User Characteristics & Personas

The users of **TestSplit** are primarily technical professionals who work with Continuous Integration (CI) pipelines and automated testing. These include software developers, DevOps engineers, QA engineers, and technical leads who maintain CI workflows and test infrastructure.

User Expertise:

Most users are expected to have a background in computer science or software engineering. They will be familiar with version-control systems and CI/CD tools such as GitHub Actions and GitLab CI that use JUnit testing frameworks.

Users can be grouped into two main categories:

- **Beginner Users:** Junior developers or team members with limited experience in CI optimisation. They may understand how to run automated tests, but have little knowledge of test profiling or performance tuning. These users benefit from simple CLI commands, clear documentation, and visual dashboards with minimal configuration.

- **Advanced Users:** DevOps engineers and senior developers experienced in tuning CI pipelines, analysing test performance, and customising CI configurations. These users seek deeper insight into test distribution and more control over how tests are split and scheduled.

User Objectives:

From the user's perspective, **TestSplit** is a tool to:

- Reduce CI pipeline duration by distributing tests more effectively across parallel jobs.
- Profile and analyse test execution times to identify slow or unstable tests.
- Generate configuration files that integrate directly with existing CI workflows.
- Compare baseline and optimised test distributions to validate performance improvements.
- Increase team productivity by shortening feedback cycles during development.

Usage Context:

Users interact with TestSplit primarily through a command-line interface (CLI) and a local web dashboard. The CLI parses JUnit test results, profiles execution times, and generates optimised CI configuration files. The dashboard loads locally generated profiling data to visualise performance metrics, test-suite balance, and historical trends between runs.
Both tools operate on the same development machine or within a reproducible containerised environment, enabling consistent results when profiling, optimising, or validating CI pipelines.

TestSplit integrates with existing CI systems, such as GitHub Actions or GitLab CI, without requiring changes to tests or the repository structure.

Desirable Features (Wish List):

In future phases, users may want:

- Support for additional test formats (e.g., Jest).
- CI configuration generation for other systems (e.g., Jenkins, CircleCI).
- Flaky-test detection and alerting.
- Advanced visualisations (e.g., job timelines, test-heatmaps).

These enhancements are outside the scope of the initial implementation.

2.3 Operational Scenarios

Simplified Use Cases:

The following simplified use cases summarise the core functional capabilities of the TestSplit system. Each use case represents a key interaction or outcome that the system provides to its users.

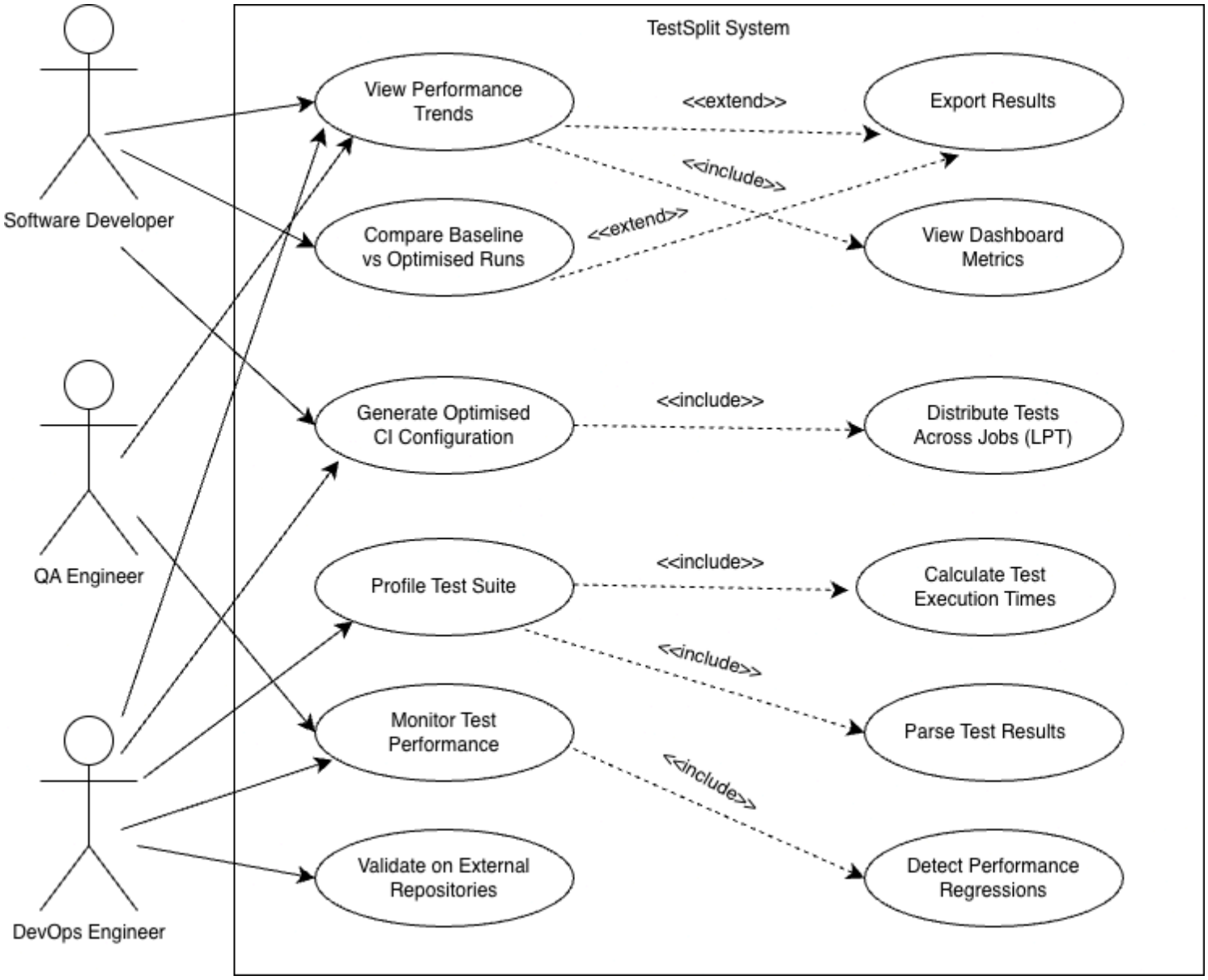


Figure 1: Use Case Diagram

Profile Test Suite:

- Analyse unit-test results to measure individual test durations and identify performance bottlenecks.
- Generate recommendations for balanced parallel distribution based on measured execution times.

Generate Optimised Configuration:

- Automatically generate GitHub Actions or GitLab CI YAML configuration files that define parallel job distribution to improve CI efficiency.

View Performance Trends:

- Load profiling data into the local dashboard to visualise performance improvements and runtime trends across multiple CI runs.

Compare Baseline vs Optimised Runs:

- Evaluate performance before and after applying TestSplit's recommendations by calculating total runtime reduction achieved in CI pipelines.

Validate on External Repositories:

- Apply TestSplit to open-source projects to assess its effectiveness and consistency in different CI environments.

Export Results:

- Export profiling and analysis data in JSON or CSV formats for reporting, documentation, and research.

Scenario A – Initial Test Optimisation (Software Developer):

User: Alex, a software developer working on a mid-sized open-source project.
Objective: Reduce the duration of the project's unit-test stage in the Continuous Integration (CI) pipeline.

Environment:

- **User Device:** Developer laptop or workstation.
- **System Interface:** TestSplit CLI.
- **Dependencies:** JUnit test framework, GitHub Actions CI.

Steps:

1. Alex runs the project's test suite locally using the existing JUnit framework, which produces results in XML format.
2. Using the CLI, Alex profiles the exported test results to analyse execution times and identify slower tests.
3. TestSplit processes the data and recommends dividing the test suite into four parallel jobs, estimating a potential two to three-fold reduction in total execution time compared with the existing single-job setup.
4. Alex generates an optimised GitHub Actions configuration file using the CLI, which includes the proposed parallel job distribution.
5. The generated configuration file is committed to the repository and pushed to GitHub.
6. On the next CI run, tests execute in parallel using the optimised configuration.
7. The total runtime decreases from approximately 9 minutes to 3.5 minutes, improving feedback time for pull requests and overall productivity.

Scenario B – Performance Review and Regression Detection (DevOps Engineer):

User: Jordan, a DevOps engineer responsible for maintaining CI/CD infrastructure.
Objective: Monitor test performance over multiple profiling runs and detect regressions in test distribution.

Environment:

- **User Device:** Workstation or laptop running Dockerised TestSplit environment.
- **System Interface:** TestSplit CLI and local dashboard.
- **Dependencies:** JUnit test framework, GitHub Actions or GitLab CI.

Steps:

1. Jordan profiles test results across several CI runs using the CLI and stores the generated JSON outputs.
2. The local dashboard loads these profiling JSON files and displays test durations, job balance, and predicted speed-ups.
3. The trend chart shows improvement from an average 2.8× speed-up to 3.4× after recent optimisations.
4. Jordan notices a runtime imbalance, suggesting a newly added test is taking longer than expected.

5. The dashboard highlights the test responsible for the imbalance.
6. After optimising that test, Jordan re-runs TestSplit to verify that the workload distribution and total runtime have stabilised.

This workflow demonstrates TestSplit's ability to identify regressions and verify improvements in a reproducible, static environment.

Scenario C – Flaky Test Detection (Future Enhancement):

User: Priya, a QA engineer responsible for ensuring test reliability.
Objective: Identify and monitor flaky tests that show inconsistent execution times or intermittent failures.

This feature would analyse multiple profiling runs to identify tests with high variance in duration. A stability score would be assigned to each test, helping engineers locate nondeterministic tests that cause unreliable performance.
This feature is outside the scope of the initial implementation but will be explored as a potential Phase B enhancement.

2.4 Constraints

Technical Constraints:

- **Supported Frameworks:** TestSplit supports any testing framework that produces JUnit-compatible XML results.
- **Runtime Environment:** The CLI requires Node.js 18 or later.
- **Containerisation:** Profiling and validation are executed inside a Docker container to maintain a consistent runtime environment across tests and ensure reproducible results.
- **Local Storage Access:** The CLI must be able to read and write local profiling data for comparison between runs.
- **Frontend Compatibility:** The dashboard requires a modern browser with ES6+ support (e.g., Chrome, Firefox, Edge, Safari).

Platform Constraints:

- **CI Integration:** TestSplit generates configurations for GitHub Actions and GitLab CI, both of which support parallel job execution.
- **Parallelisation Requirements:** The target CI platform must allow test sharding or file-pattern-based job distribution.
- **Operating Systems:** The CLI supports macOS (primary development), Linux, and Windows.

Resource Constraints:

- **Development Environment:** A machine with at least 4 CPU cores and 8 GB RAM is recommended.
- **Container Baseline:** Validation runs are executed under a controlled, steady-state Docker configuration to ensure performance consistency and fairness between baseline and optimised runs.
- **Storage Requirements:** Approximately 10 MB per 1,000 tests is required for storing historical profiling data/
- **Network Connectivity:** Internet access is only needed for package installation or CI execution. All profiling and optimisation run locally.

Security and Privacy Constraints:

- All profiling and test metadata remain local to the user's environment.
- No data is transmitted externally or stored on external servers.
- TestSplit does not access or modify source code beyond reading JUnit XML results.

Algorithmic Constraints:

- **Independence Assumption:** The LPT scheduling algorithm assumes that all tests can execute independently, without requiring a dependency ordering.
- **Runtime Stability:** Profiling is performed in a containerised environment to reduce variability from host hardware.
- **Historical Data Dependency:** The first profiling run establishes baseline timings used to refine later distributions.

3. Functional Requirements (FRs)

Requirement 1 (FR1): Parse Test Results from Standard Formats

Description:
The system must parse test result files exported from JUnit (versions 4 and 5). It must extract relevant metadata including test names, durations, and execution status (passed, failed, or skipped). Parsed data forms the foundation for all subsequent profiling and optimisation processes.

Criticality:
High: This is the first and most essential step in the workflow.

Technical Issues:

- Minor variations in XML structure between JUnit 4 and 5.
- Handling malformed or incomplete result files.
- Managing nested or parameterised test cases.

Mitigation Strategies:

- Implement strict XML schema validation with fallback error handling.
- Ignore or flag malformed entries without halting execution.
- Design extensible parser modules to support other test frameworks later (e.g., Jest).

Dependencies:
This is the first operation in the system pipeline. Its output provides structured input to the profiling engine (FR2).
Errors or inconsistencies here can cascade into inaccurate profiling or grouping results (FR3).

Requirement 2 (FR2): Test Profiling Engine

Description:
The system must calculate each test's execution time and store profiling results for reuse. This includes calculating averages across multiple runs and detecting anomalous durations that may indicate instability or environmental issues.

Criticality:
High: Accurate profiling is essential for balanced parallelisation and reliable speed-up measurements.

Technical Issues:

- Inconsistent timing data across hardware or runs.
- Missing or zero-duration entries.
- Handling flaky or intermittent test behaviours.

Mitigation Strategies:

- Normalise timings using multiple-run averaging or statistical smoothing.

- Flag tests with unstable or zero durations for user review.
- Run profiling inside a containerised environment to ensure reproducibility.

Dependencies:
Requires structured test data from the parser (FR1).
Outputs profiling results in a consistent format, consumable by the grouping algorithm (FR3), and stores them locally (FR5).
Inaccurate profiling data can lead to poor LPT scheduling outcomes or misleading dashboard metrics (FR7/FR9).

Requirement 3 (FR3): Parallelisation and Test Grouping

Description:
The system must distribute tests into balanced parallel jobs using the Longest Processing Time (LPT) scheduling algorithm. The goal is to minimise overall CI runtime by ensuring that each job has a similar total execution duration.

Criticality:
High: This is the core optimisation mechanism that delivers measurable improvements in CI performance.

Technical Issues:

- Trade-off between job count and runtime.
- Variation in test durations between runs.
- Impact of missing or incomplete profiling data.

Mitigation Strategies:

- Allow users to specify the number of desired parallel jobs.
- Automatically recalculate groupings as new profiling data becomes available.
- Re-run the algorithm periodically to account for regressions or codebase changes.

Dependencies:
Requires accurate timing data from the profiling engine (FR2).
Outputs balanced job groups that are consumed by the CI configuration generator (FR4).
Relies on consistent historical data stored locally (FR5) for iterative improvement.

Requirement 4 (FR4): CI Configuration Generation

Description:
The system must generate valid YAML configuration files for CI systems such as GitHub Actions and GitLab CI. These files define the job matrices and specify which test groups will execute within each job.

Criticality:
High: This enables direct integration of TestSplit's optimisations into real CI workflows.

Technical Issues:

- Syntax and structure differences between CI platforms.
- Potential for invalid YAML causing failed builds.
- Platform-specific limits on job count or concurrency.

Mitigation Strategies:

- Maintain separate CI templates for each supported platform.

- Validate generated YAML files against the platform schema before saving.
- Provide a “dry-run” option to preview configurations before writing to disk.

Dependencies:
Uses grouped test data from FR3.
Requires consistent naming and structure to match CI runner expectations.
Outputs configuration files that developers commit directly to their repositories.

Requirement 5 (FR5): Local Data Storage

Description:
TestSplit must persist profiling and grouping data locally to allow reuse between runs and enable trend analysis. This ensures users can view past performance metrics and verify improvements without re-profiling each time.

Criticality:
High: Local persistence supports both usability and performance.

Technical Issues:

- Maintaining consistency between runs with different commits.
- Handling renamed or refactored tests.
- Preventing excessive disk-space growth over time.

Mitigation Strategies:

- Use lightweight file-based storage (JSON or SQLite) with indexing for faster lookups.
- Tag test records with commit hashes or UUIDs.
- Rotate or compress old profiling data automatically after a configurable number of runs.

Dependencies:
Consumes profiling output from FR2 and grouping data from FR3.
Feeds trend visualisation modules in the dashboard (FR7) and historical analysis (FR9).

Requirement 6 (FR6): Command-Line Interface (CLI)

Description:
A command-line interface must be provided for developers to run profiling, generate CI configurations, and view summary results. The CLI acts as the main control surface for integrating TestSplit into development workflows.

Criticality:
High: Primary user interaction point.

Technical Issues:

- Complex argument parsing and nested commands.
- Cross-platform compatibility across macOS, Linux, and Windows.
- Maintaining readability of terminal output.

Mitigation Strategies:

- Implement with Commander.js and chalk for structured output.
- Provide in-command help (--help) and usage examples.

- Use clear colour-coded status messages for success, warning, and error states.

Dependencies:
Orchestrates FR1–FR5 in sequence.
Reads/writes from local storage (FR5).
Operates independently from the dashboard (FR7).

Requirement 7 (FR7): Local Dashboard for Historical Metrics

Description:
A local dashboard must display profiling metrics and job-distribution results across multiple runs. Users can review total runtimes, test balances, and speed-up metrics through visual charts and tables.

Criticality:
Medium: Not essential for functionality but critical for validation and demonstration.

Technical Issues:

- Efficiently rendering large datasets in the browser.
- Ensuring accurate visualisation of time-series metrics.
- Handling multiple datasets from different projects.

Mitigation Strategies:

- Implement React + Recharts or Chart.js with data virtualisation.
- Cache and paginate local datasets to reduce memory usage.
- Allow import/export of saved JSON profiles for reuse.

Dependencies:
Uses profiling data from FR2 and historical storage from FR5.
Displays grouping results from FR3 and configuration outcomes from FR4.

Requirement 8 (FR8): Containerised Execution Environment

Description:
The system must execute profiling and validation inside a Docker container to ensure consistent runtime conditions. This prevents hardware differences from skewing results and enables reproducible benchmarking.

Criticality:
Medium: Not user-facing but essential for reliable performance evaluation.

Technical Issues:

- Managing container setup across operating systems.
- Balancing CPU and memory allocation to simulate realistic CI environments.
- Version drift between host and container images.

Mitigation Strategies:

- Provide a predefined Dockerfile and CLI alias to simplify setup.
- Use identical container configurations for baseline and optimised runs.
- Version-lock dependencies and image tags to maintain reproducibility.

Dependencies:
Runs all profiling (FR2), grouping (FR3), and validation processes inside a stable environment.
Supports consistent performance comparisons and future regression analysis (FR9).

Requirement 9 (FR9): Historical Trend Analysis

Description:
The system must analyse and visualise historical performance trends, allowing users to identify regressions, flaky tests, or newly introduced bottlenecks.

Criticality:
Medium: Supports long-term performance evaluation and reliability tracking.

Technical Issues:

- Mapping test identifiers across commits.
- Handling large amounts of time-series data.
- Detecting statistically significant regressions.

Mitigation Strategies:

- Store incremental deltas rather than complete histories.
- Implement smoothing filters to highlight genuine anomalies.
- Allow users to tag or group related tests manually.

Dependencies:
Built on profiling data (FR2) and local storage (FR5).
Visualised through the dashboard (FR7).
Runs on top of the containerised environment (FR8) for stable comparisons.

4. System Architecture

The TestSplit system uses a modular monorepo architecture that separates command-line operations, shared analysis logic, and data visualisation. The system consists of five primary modules: the CLI, Parsers, Algorithm, Generators, and Frontend. These modules communicate via shared TypeScript models and local storage, enabling the system to operate effectively offline while remaining compatible with CI environments such as GitHub Actions and GitLab CI.

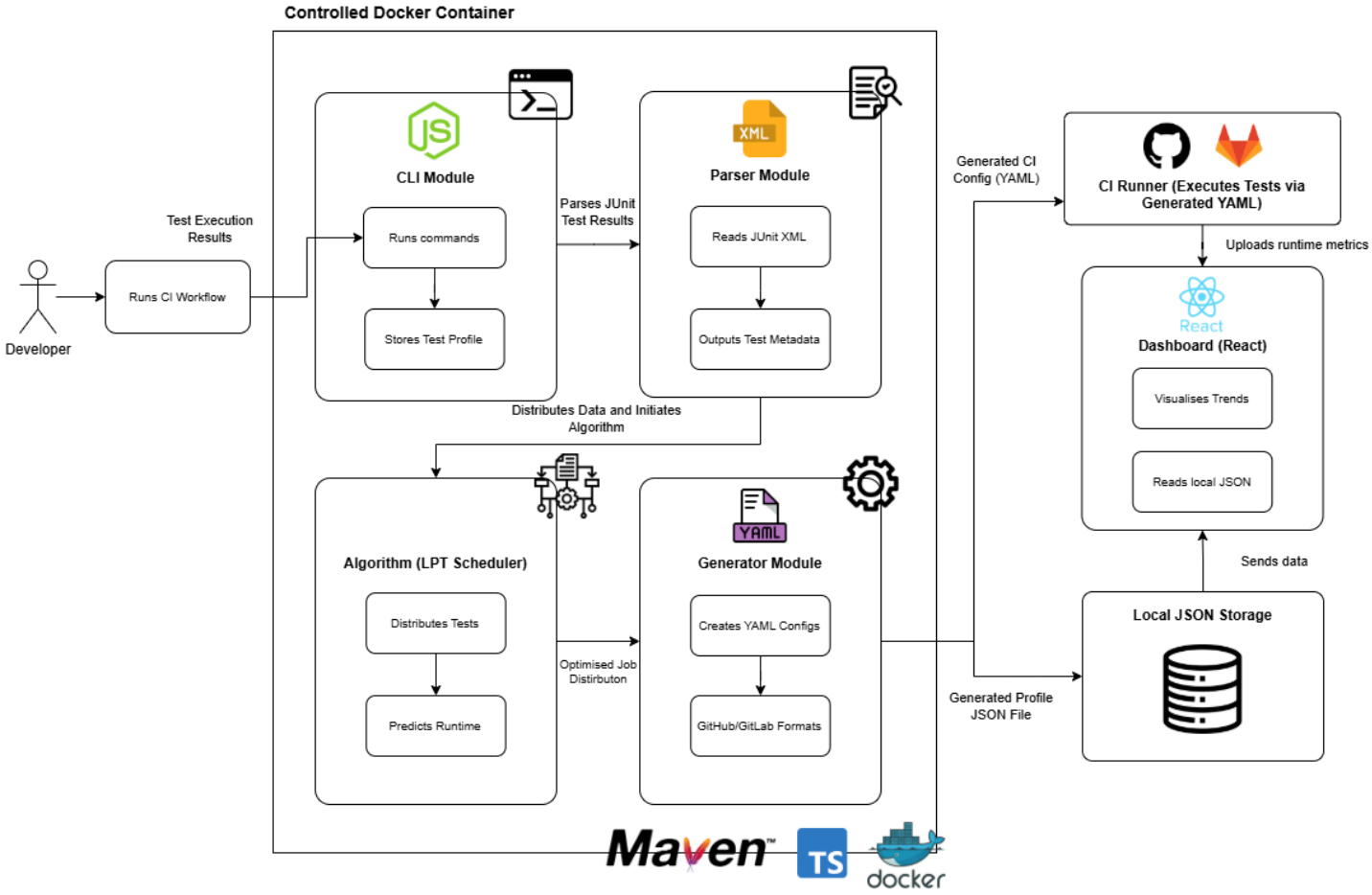


Figure 2: System Architecture Diagram

4.1 System Modules and Functions

Module: CLI (packages/cli/)

- Acts as the primary user interface for interacting with the system.
- Parses command-line arguments using Commander.js.
- Invokes the parser, algorithm, and generator modules to perform profiling, scheduling, and configuration generation.
- Formats terminal output as tables, JSON, or HTML.
- Manages local data persistence for profiling history.

Includes:

- commands/profile.ts Implements profiling functionality.
- commands/generate-config.ts Generates CI configuration files.
- commands/validate.ts Runs validation checks on generated configurations.
- utils/formatting.ts Handles terminal formatting and output presentation.
- storage/LocalStorage.ts Handles reading/writing persistent profiling data.

Module: Parsers (packages/shared/src/parsers/)

- Responsible for reading and interpreting **JUnit XML test result files**.
- Extracts key metadata, including test name, duration, and status.
- Handles malformed or incomplete input gracefully to prevent analysis failures.

Includes:

- JUnitXMLParser.ts: Core XML parsing logic.
- TestParser.ts: Defines shared parser interfaces and data types.

Module: Algorithm (packages/shared/src/algorithm/)

- Implements the **Longest Processing Time (LPT)** scheduling algorithm for parallelisation.
- Calculates job distributions and predicted time savings.
- Generates metrics such as total duration, mean balance ratio, and predicted speed-up.

Includes:

- LPTScheduler.ts: Implements LPT scheduling logic (O(n log n) complexity).
- statistics.ts: Performs statistical analysis and performance evaluation.
- types.ts: Defines shared interfaces for test and job data structures.

Module: Generators (packages/shared/src/generators/)

- Creates valid **CI configuration files** in YAML format.
- Validates output syntax before saving to ensure compatibility with CI platforms.
- Supports generation for **GitHub Actions** and **GitLab CI**.

Includes:

- GitHubActionsGenerator.ts: GitHub-specific configuration.
- GitLabCIGenerator.ts: GitLab-specific configuration.
- ConfigGenerator.ts: Defines a shared interface for generators.

Module: Frontend (packages/frontend/)

- Provides a **local React dashboard** for visualising profiling and optimisation metrics.
- Displays performance trends, job balance charts, and execution-time distributions.
- Allows users to upload JSON results or view cached run data.

Includes:

- components/FileUpload.tsx: File upload and import handler.
- components/TrendChart.tsx: Visualises time-series and distribution trends.
- components/StatsSummary.tsx: Displays aggregated test statistics.
- pages/Dashboard.tsx: Main entry page for viewing analysis results.

4.2 Interaction Flow (High-Level)

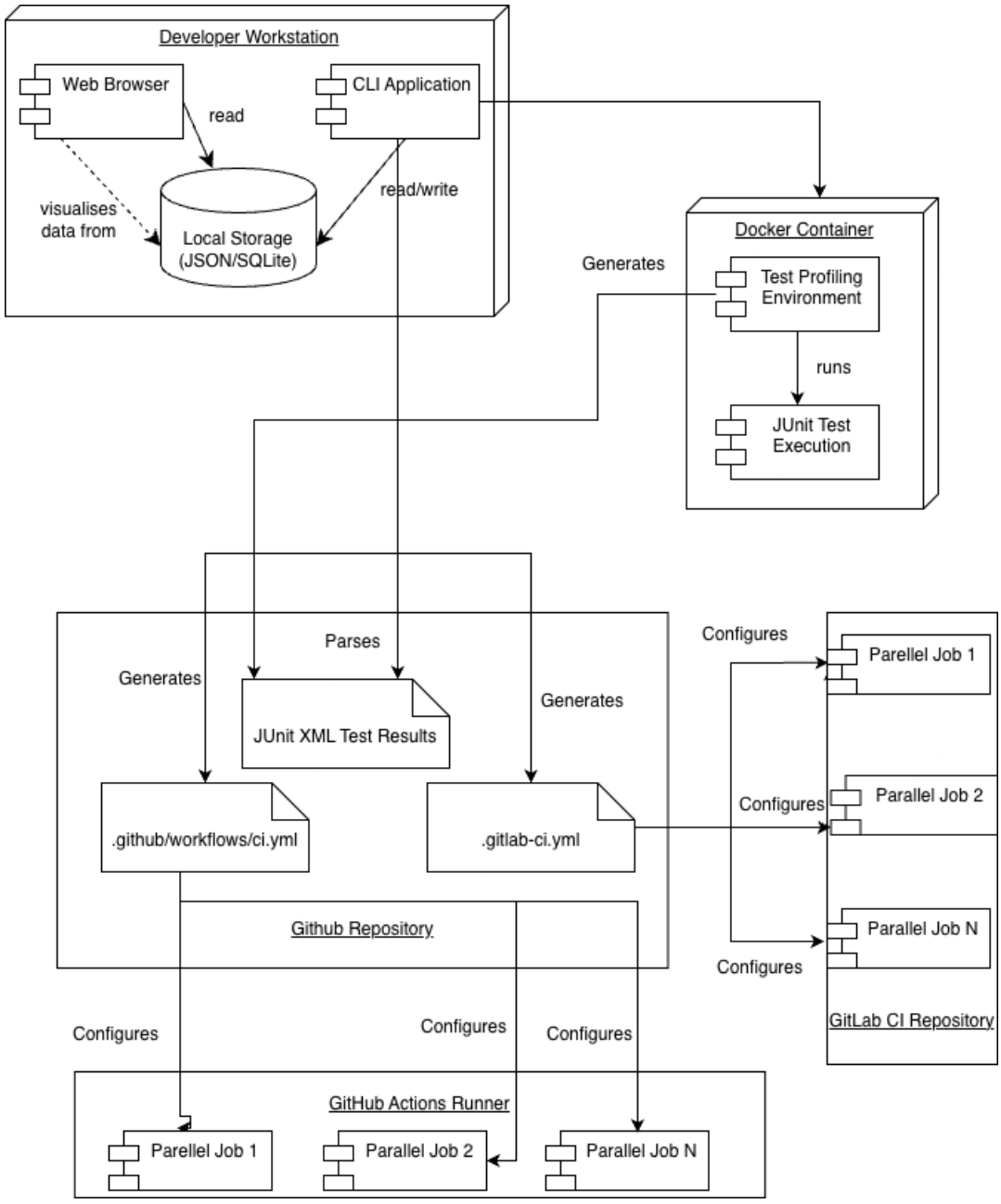


Figure 3: Deployment Diagram

Flow 1: Profile Test Suite

1. The user runs the testsplit profile from the CLI.
2. The **Parsers** module reads JUnit XML test results from the project's target/surefire-reports/ directory.
3. Extracted test durations and metadata are passed to the **Algorithm** module.
4. The **LPT Scheduler** determines the optimal distribution of jobs across a configurable number of jobs.
5. The CLI stores profiling data locally (e.g., ~/.testsplit/history.json).
6. Results are displayed in the terminal and optionally exported for dashboard analysis.

Flow 2: Generate CI Configuration

1. The user runs testsplit generate-config --platform github --jobs 4.
2. The **Algorithm** output is passed to the **Generators** module.
3. A valid YAML configuration file is produced and validated.

4. The configuration is written to .github/workflows/ci.yml or equivalent for GitLab.

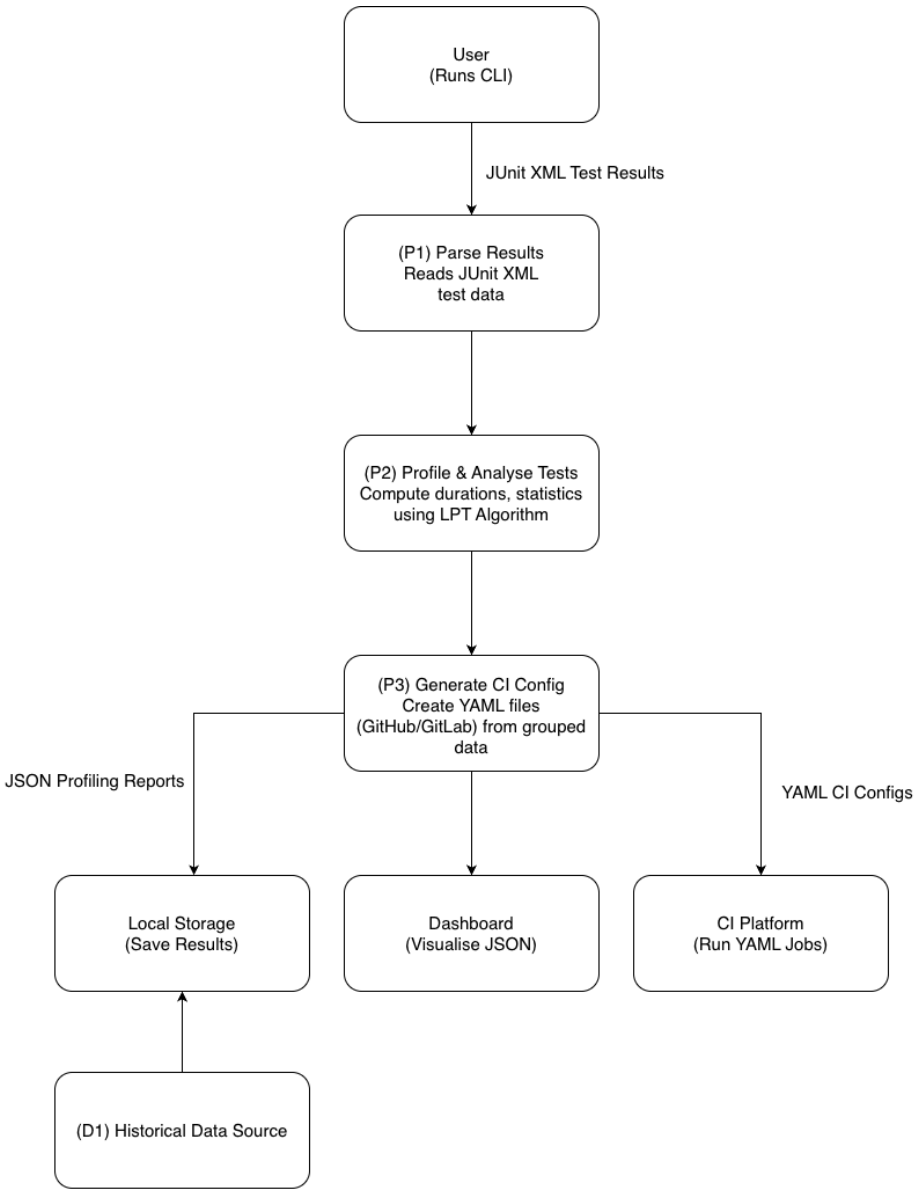


Figure 4: Data Flow Diagram

Additional flows for visualisation and validation are illustrated in Section 4.4 with sequence diagrams.

4.3 Environmental Context

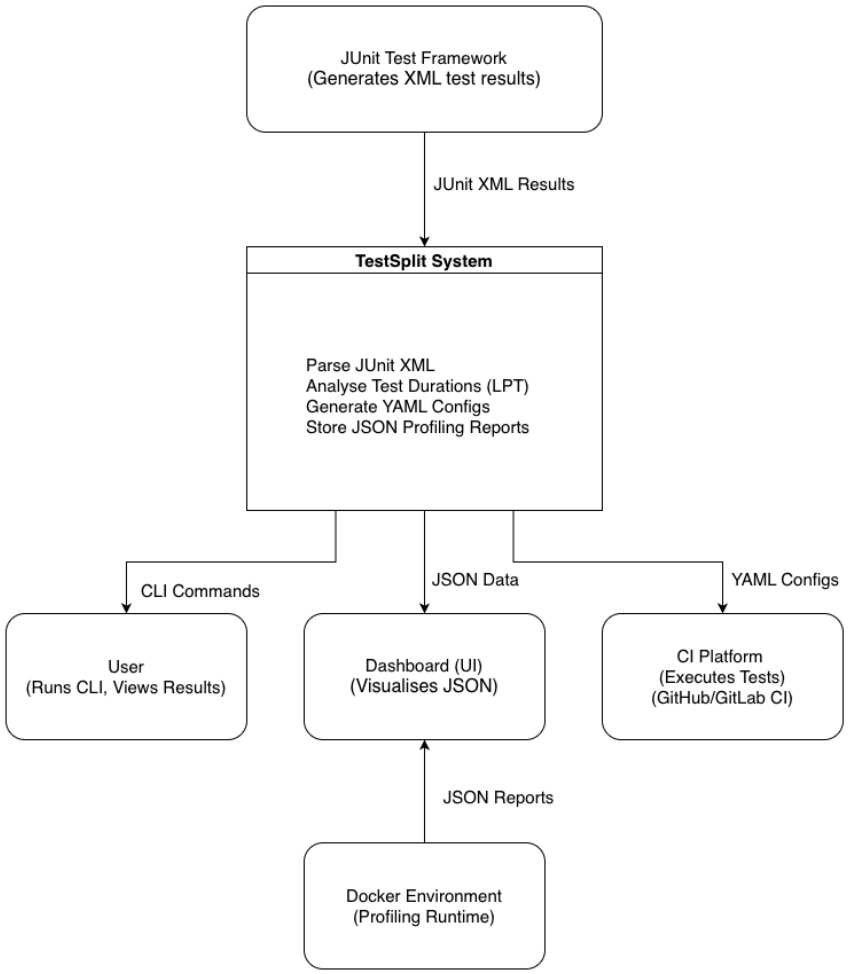


Figure 5: System Context Diagram

Docker Environment:

- All profiling, parsing, and validation are executed inside a Docker container for reproducibility.
- Ensures consistent resource allocation (e.g., CPU cores, memory) and eliminates hardware bias.
- Defines a uniform baseline for comparing single-threaded vs. multi-threaded CI job scheduling.

CLI Environment:

- Runs locally on developer workstations (macOS, Linux, or Windows).
- Requires Node.js 18+ and npm 8+.
- Reads/writes profiling data from the local filesystem.
- Interfaces with the Docker container for stable, reproducible runs.

CI Environment (Generated Configurations):

- Executes on **GitHub Actions** or **GitLab CI** runners.
- Requires test frameworks supporting sharding or filtering.
- Uses standard runner permissions for dependency installation and test execution.
- Generated configurations can be validated in the same containerised environment for accuracy.

Dashboard Environment:

- Runs locally within a modern web browser (Chrome, Firefox, Edge, Safari).

- No network access required.
- Accessed via a local development server started by the CLI (e.g., testsplit dashboard).
- Visualises historical profiling data and performance trends.

This structure ensures a reproducible, containerised environment for profiling and testing while maintaining developer accessibility. The CLI performs all profiling and configuration generation, the container ensures consistency across environments, and the dashboard enables intuitive performance analysis.

4.4 Reference Diagrams

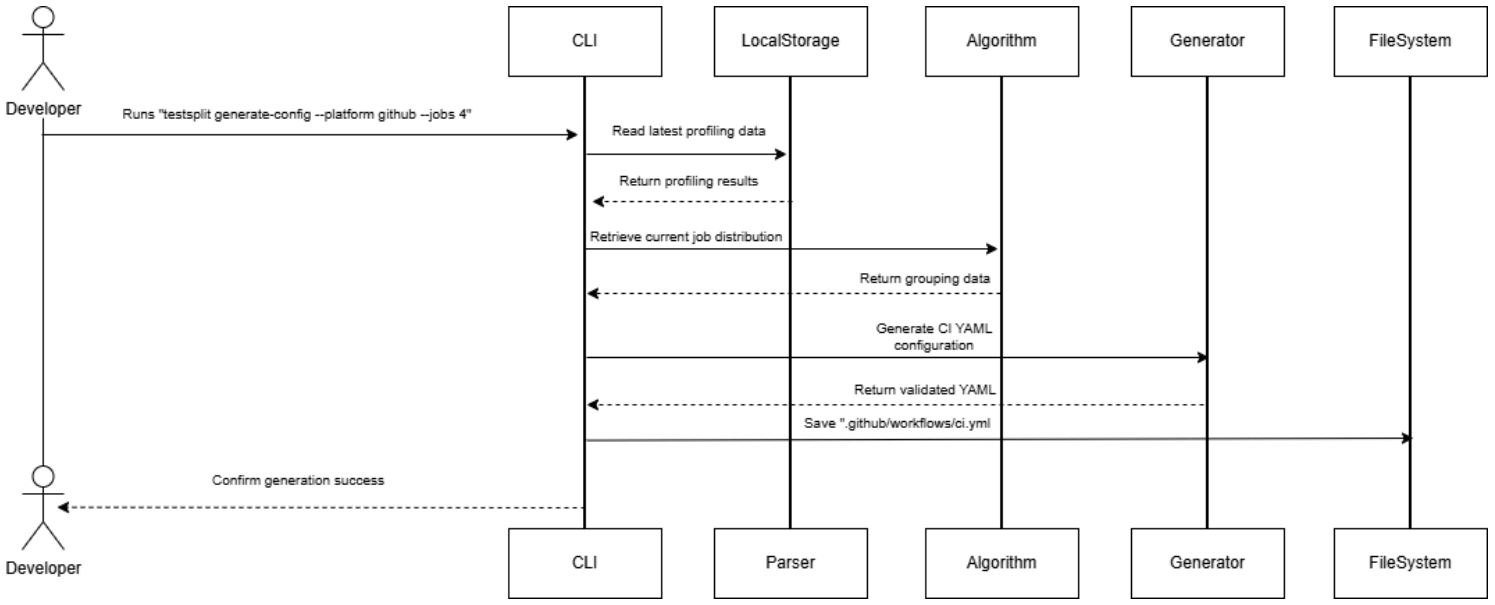


Figure 5: Sequence Diagram - Configuration Generation

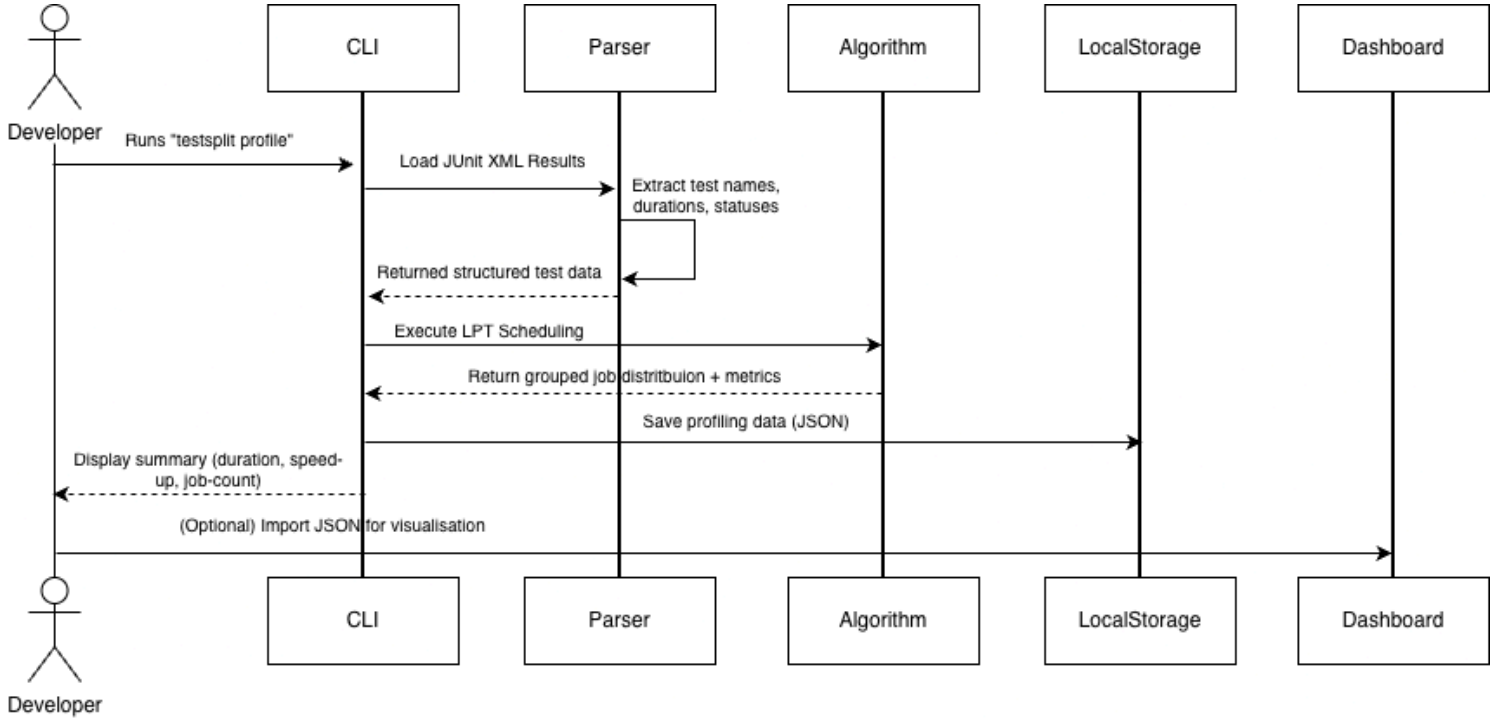


Figure 6: Sequence Diagram - Dashboard Visualisation

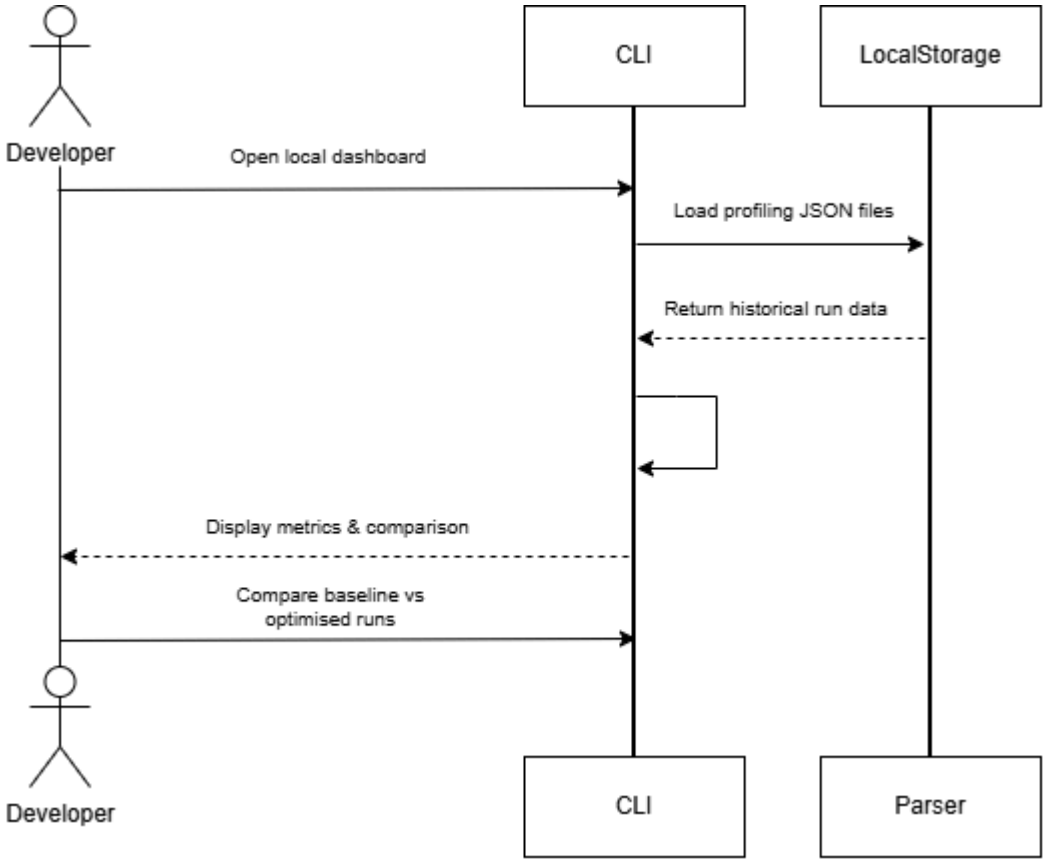
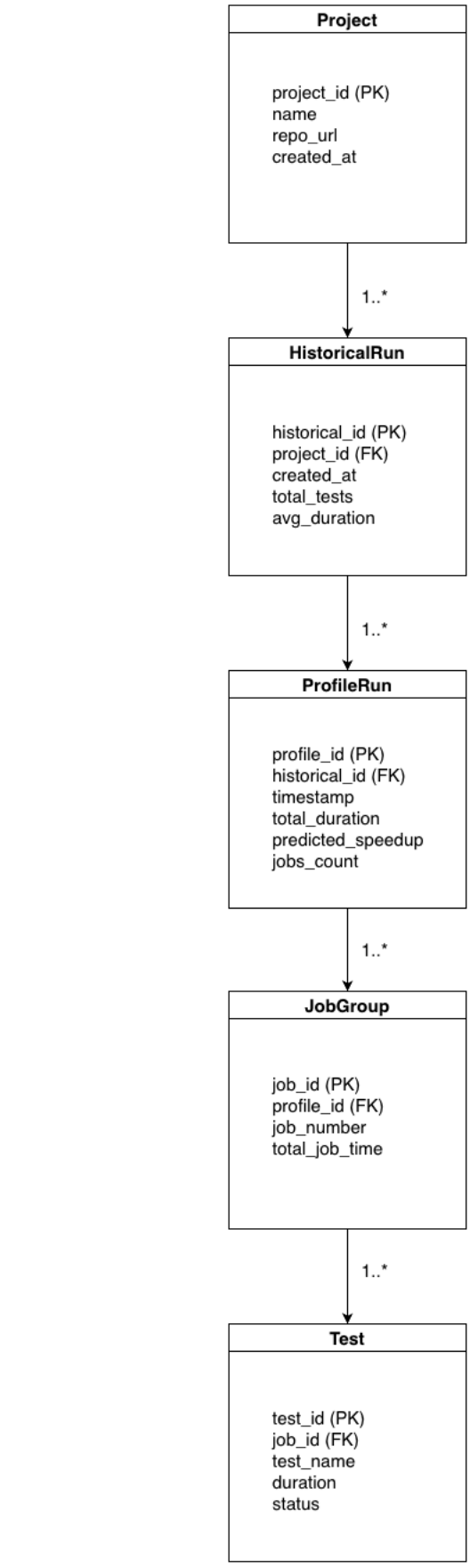


Figure 7: Sequence Diagram - Run Comparison

5. High-Level Design & Diagrams

5.1 Data Model



PK - Primary Key
FK - Foreign Key
1..* - One-to-Many relationship

Figure 8: ER Diagram

5.1.1 Job Distribution Schema

The Job Distribution Schema defines how individual tests are grouped into parallel execution jobs following the Longest Processing Time (LPT) scheduling algorithm. It represents the output of the LPTScheduler module and is used by the ConfigGenerator to produce platform-specific YAML configurations for Continuous Integration (CI) systems.

Each job group contains the set of tests assigned to it, the total execution duration, and the balance metrics used to evaluate the distribution's efficiency.

Structure:

- **jobId (string):** Unique identifier for the generated job group.
- **assignedTests (List<TestResult>):** Collection of test cases allocated to this job.
- **totalTime (float):** Sum of all test durations within the job, in seconds.
- **balanceRatio (float):** Ratio used to measure how evenly workloads are distributed across jobs.
- **calculateBalance():** Method that computes the uniformity metric used to assess load balance.

Usage: This schema is written to disk by the LocalStorage module, consumed by the ConfigGenerator, and visualised in the Dashboard as part of the profiling output.

5.1.2 Profile Schema

The Profile Schema represents the data produced during a single profiling run. It captures individual test results, aggregated metrics, and environment metadata, forming the foundation for later analysis and CI optimisation.
This schema is generated by the Profiler module and stored locally in JSON format.

Structure:

- **id (string):** Unique identifier for the profiling run (UUID or timestamp).
- **timestamp (Date):** Time the profiling session was executed.
- **results (List<TestResult>):** All parsed tests, including duration and status.
- **totalDuration (float):** Total runtime of all tests in the profiling session.
- **averageDuration (float):** Mean runtime across all tests.
- **suggestedJobs (int):** Number of parallel jobs recommended by the LPT scheduler.
- **environment (object):** Metadata describing runtime conditions (OS, container image, CPU count).
- **save()/load():** Methods that handle writing and retrieving profiles from local storage.

Usage: The Profile Schema is the core input for:

- The LPTScheduler, which computes optimal test grouping.
- The Dashboard, which visualises results through charts and summaries
- The Historical Data Schema which aggregates multiple profiles for regression analysis.

5.1.3 Historical Data Schema

The Historical Data Schema maintains an aggregated record of multiple profiling sessions to enable long-term performance tracking and regression detection. It forms the basis for trend visualisation and comparison in the Dashboard module.

Each historical record references one or more Profile objects, preserving data continuity across commits, profiling sessions, and configuration updates.

Structure:

- **projectId (string):** Identifier for the project or repository under analysis.
- **profiles (List<Profile>):** Array of stored profile records ordered chronologically.
- **averageSpeedUp (float):** Mean speed-up ratio across recorded runs.
- **balanceHistory (List<float>):** Historical balance metrics across job distributions.
- **regressionFlags (List<int>):** Indices marking runs with detected slowdowns or anomalies.
- **lastUpdated (Date):** Timestamp of the latest profiling session.
- **compareRuns():** Method that computes deltas between baseline and optimised profiles.
- **exportReport():** Generates summary output for the dashboard view.

Usage: This schema is used exclusively by the Dashboard for plotting performance trends and identifying regressions over time.

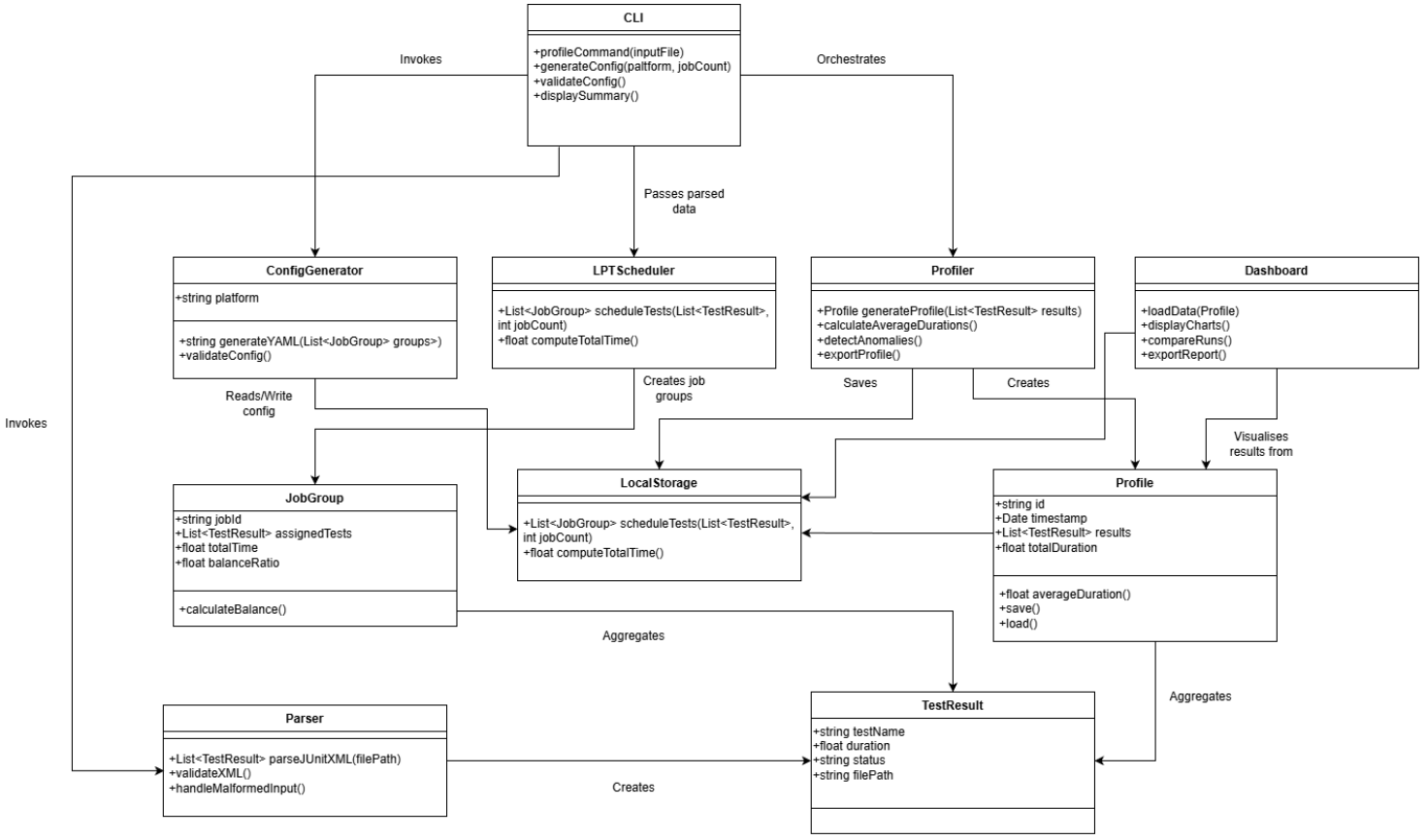


Figure 9: High-Level Class Diagram

5.3 Non-Functional Design Choices

Scalability:

- **CLI:** Single-user operation with no scalability concerns.
- **Algorithm:** O(n log n) complexity, capable of handling 10 000 + tests in < 100 ms.

- **Storage:** Local JSON or SQLite storage scales to 100 runs per project before rotation.
- **Dashboard:** Client-side rendering scales with browser memory (~10 MB limit).
- **Containerisation:** All profiling and validation run inside Docker for reproducible resource limits and stable performance measurements.

Performance:

- **JUnit XML Parsing:** Streaming parser is efficient for large files (> 10 MB).
- **LPT Algorithm:** In-memory operation with no database queries.
- **YAML Generation:** Template-based output completes in <10 ms for typical configurations.
- **Chart Rendering:** Virtual scrolling efficiently supports > 1,000 data points.
- **Container Overhead:** Negligible (< 2 %) under identical host conditions.

Reliability:

- **File I/O:** Atomic writes using temporary files and rename operations.
- **Validation:** YAML schema validation before saving configuration.
- **Error Handling:** Graceful degradation (continues without historical data if write fails).
- **Testing:** Target unit-test coverage 85%+.
- **Docker Execution:** Provides a stable, repeatable profiling environment independent of host hardware.

Security:

- **Local Operation:** No network communication or data uploads.
- **File Permissions:** Respect the user mask (0600 is the default for local history).
- **Input Validation:** Sanitise file paths and XML inputs to avoid injection or traversal.
- **Dashboard:** CSP headers prevent XSS; no external scripts loaded.

Usability:

- **CLI:** Includes clear help text (testsplit --help) and colour-coded output.
- **Error Messages:** Provide actionable suggestions (e.g. “Run npm test -- --json”).
- **Documentation:** README includes setup steps, examples, and troubleshooting guides.
- **Dashboard:** Responsive layout for desktop and mobile browsers.

Maintainability:

- **Monorepo Structure:** Separate packages for CLI, shared logic, and frontend.
- **TypeScript:** Static typing reduces runtime errors and improves IDE feedback.
- **Testing:** Comprehensive unit tests and integration flows for core modules.
- **CI/CD:** Automated linting, testing, and build checks on every commit.

Portability:

- **Cross-Platform:** Node.js runs on Linux, macOS, and Windows.
- **Browser Support:** Modern browsers (Chrome, Firefox, Safari, Edge).
- **CI Platforms:** Extendable factory-pattern generators for new platforms.
- **Container Support:** Docker images run on any host with Docker Engine 24 or later.

6. Testing and Evaluation Plan

6.1 Testing Goals

Primary Goals:

- **Correctness:** The LPT algorithm produces valid, balanced job distributions.
- **Accuracy:** Predicted speed-up within 10 % of actual runtime.
- **Robustness:** Handle malformed XML inputs and large test suites.
- **Performance:** Profile 10,000 tests in under 5 seconds.
- **Usability:** Clear error messages and intuitive CLI workflows.

Success Criteria:

- Unit-test coverage 85 %+.
- Integration tests pass on Linux, macOS, and Windows.
- Validation across 15 open-source JUnit repos shows an average speed-up of > 3×.
- User testing (5+ participants) achieved an average satisfaction rating of 4/5.

6.2 Test Types and Mapping

Unit Tests (packages/*/src/**/__tests__/*.test.ts)

- Parser Tests: JUnitXMLParser.test.ts – valid XML, missing attributes, malformed structure.
- Algorithm Tests: LPTScheduler.test.ts – basic distribution, edge cases, scaling.
- Statistics Tests: statistics.test.ts – mean, median, P95, balance quality.
- Generator Tests: GitHubActionsGenerator.test.ts and GitLabCIGenerator.test.ts – YAML validation.
- CLI Tests: profile.test.ts – argument handling; generate-config.test.ts – file output and dry-run mode.

Property-Based Tests (packages/shared/src/algorithm/__tests__/*.property.test.ts)

- All tests are assigned exactly once (no duplicates).
- Job totals sum to the overall duration.
- Increasing job count improves or maintains balance.
- Deterministic output for identical input.

Integration Tests (tests/integration/*.test.ts)

- profile-flow.test.ts – end-to-end profiling validation.
- generate-config-flow.test.ts – CI config generation and validation.
- historical-storage.test.ts – read/write profiling data consistency.

E2E Tests (using Playwright in tests/e2e/*.spec.ts)

- upload-file.spec.ts – upload JSON and render charts.
- trend-visualization.spec.ts – chart interaction and export.
- responsive.spec.ts – layout tests on mobile and tablet viewports.

Performance Tests (tests/performance/benchmark.sh)

- Profile 100 tests < 100 ms.
- Profile 1000 tests < 500 ms.
- Profile 10000 tests < 5s.
- Generate config < 50 ms.

- Parse 10 MB XML in < 2 s.

Validation Tests (docs/validation-results.md)

- Execute on 15 open-source repositories inside Docker.
- Compare baseline versus optimised runs.
- Measure speed-up and prediction accuracy.
- Document results in a validation report.

6.3 Evaluation Criteria and Thresholds

Algorithm Correctness:

- All tests were assigned exactly once (100 %).
- Balance quality 90%+ for uniform workloads.
- Balance quality 75%+ for skewed workloads.

Prediction Accuracy:

- Average accuracy ≥ 90 % across validation repos.
- 80% of predictions are within 15% of the actual runtime.
- No outlier > 30 % difference (excluding known flaky tests).

Performance:

- Profile 1 000 tests < 500 ms (target < 100 ms).
- CLI commands finish in < 5s.
- Dashboard loads in < 2 s with 50 historical runs.

Code Quality:

- Unit coverage 85 %+.
- 0 critical lint errors (ESLint strict mode).
- TypeScript strict mode is enabled with no type errors.

Usability:

- User testing with an average score of 4/5.
- Profile: Generate flow completed without assistance in < 5 minutes.
- 80 % of testers rate error messages as clear.

Reproducibility:

- Makes demo runs successful on a clean Ubuntu VM.
- CI pipeline passes on all commits.

6.4 Test Data

- **Synthetic Test Suites:** Artificially generated data for controlled performance testing.
- **Real Test Results (test-data/real/):** JUnit XML outputs from open-source projects.
- **Example Applications (examples/apps/):** Sample projects bundled for demo and validation inside Docker.

7. Preliminary Schedule (GANTT)

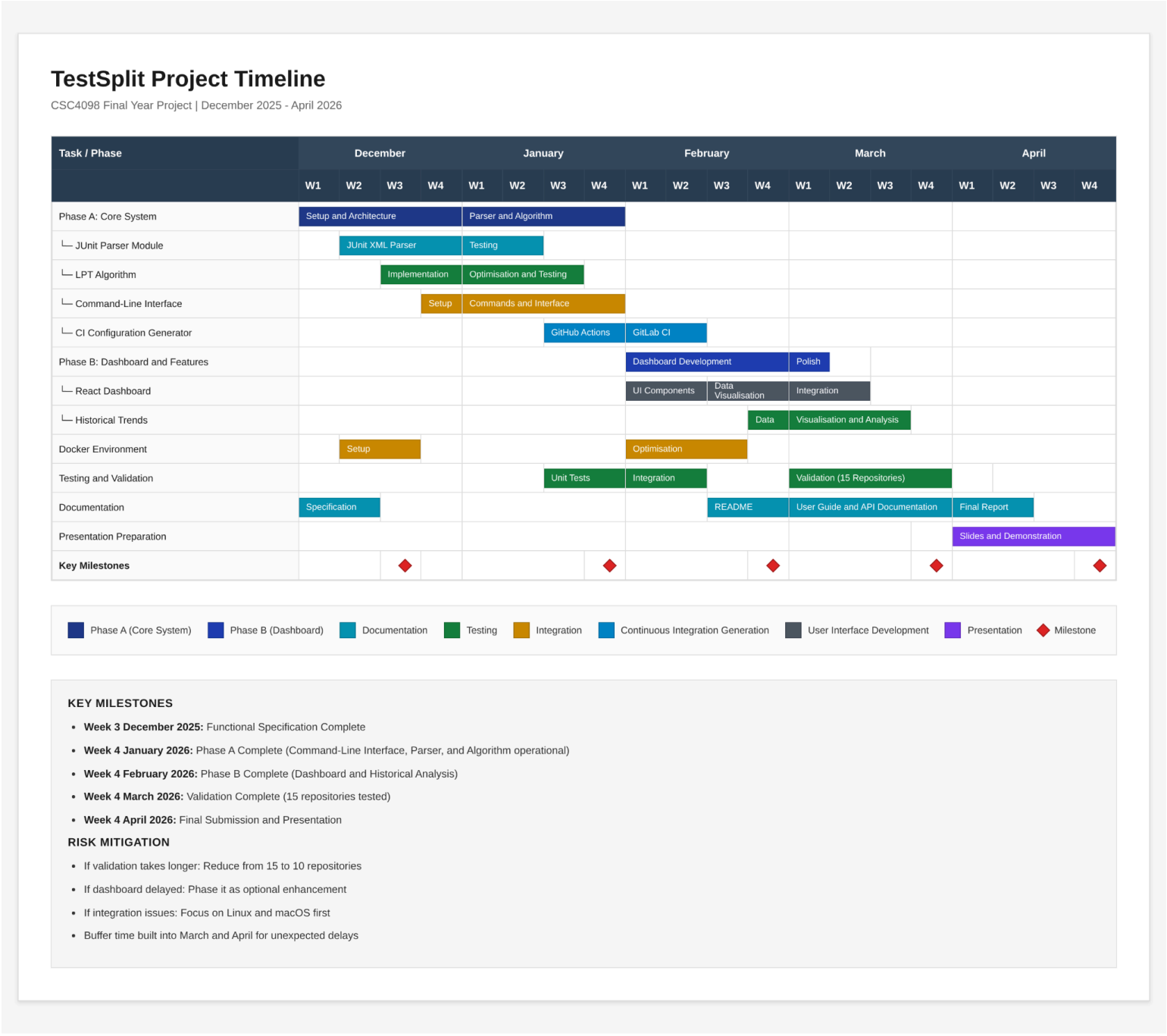


Figure 10: GANTT Chart

8. Appendices:

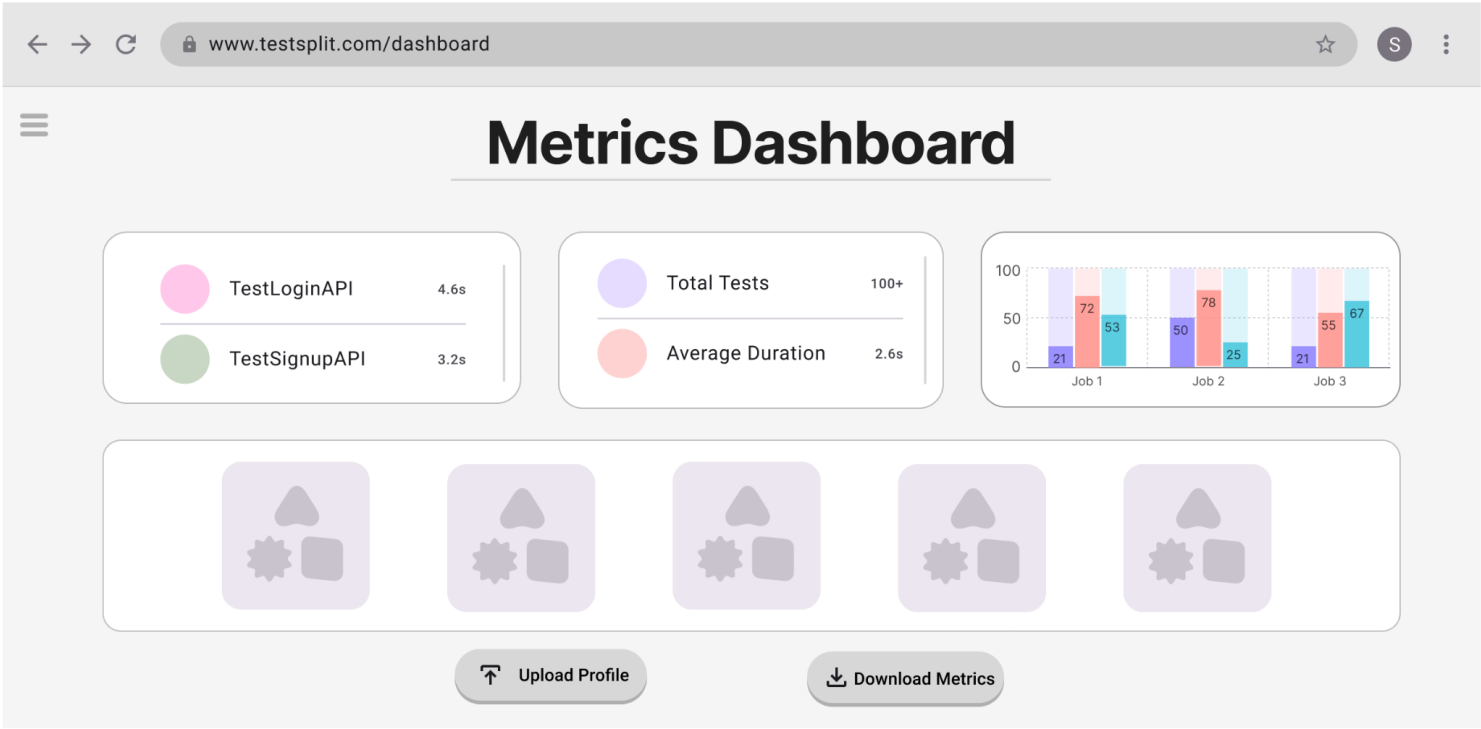


Figure 11: Wireframe for Static Dashboard

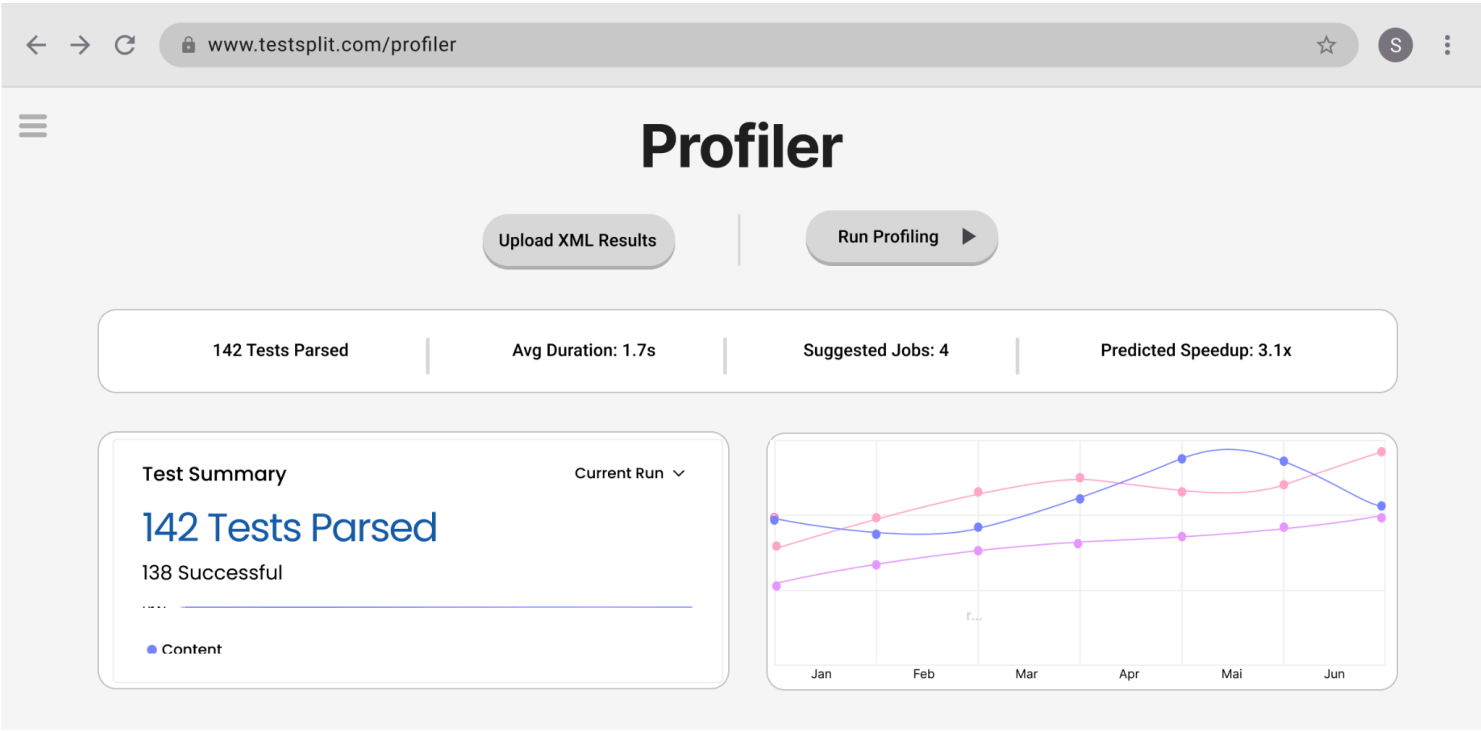


Figure 12: Wireframe for Profiler

9. References

[1] R. L. Graham, “Bounds on multiprocessing timing anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.

[2] X. Xiao, “A direct proof of the 4/3 bound of LPT scheduling rule,” *Atlantis Press*, 2017.

[3] F. Della Croce and R. Scatamacchia, “The longest processing time rule for identical parallel machines,” *Journal of Scheduling*, vol. 23, pp. 523–532, 2020.

[4] M. Dell’Amico, M. Iori, S. Martello, and M. Monaci, “Scheduling with inexact job sizes: The merits of shortest processing time first,” *arXiv preprint*, arXiv:1907.04824, 2019.

[5] GitHub Actions Documentation, *Choosing the Runner for a Job*, GitHub, 2025. [Online]. Available: https://docs.github.com/en/actions/using-github-hosted-runners/choosing-the-runner-for-a-job

[6] Docker Documentation, *Manage Compute Resources for Containers*, Docker Inc., 2025. [Online]. Available: https://docs.docker.com/engine/containers/resource_constraints/

[7] Node.js Foundation, *Worker Threads and Parallelism in Node.js*, Node.js Foundation, 2025. [Online]. Available: https://nodejs.org/api/worker_threads.html

[8] Apache Maven Project, *Maven Surefire Plugin – Generating JUnit XML Reports*, The Apache Software Foundation, 2025. [Online]. Available: https://maven.apache.org/surefire/maven-surefire-plugin/

[9] JUnit.org, *JUnit 5 User Guide – Running Tests and XML Output Format*, JUnit Team, 2025. [Online]. Available: https://junit.org/junit5/docs/current/user-guide/

[10] SonarSource Community, *Large JUnit XML Files and Analysis Performance Issues*, SonarSource, 2024. [Online]. Available: https://community.sonarsource.com/

[11] Jsoup Project, *Example Open-Source Repository for JUnit Profiling*, GitHub, 2025. [Online]. Available: https://github.com/jhy/jsoup